

Analisi Approfondita dei Cambiamenti: Confronto Evolutivo tra Sprint 0 e Sprint 1 nel Modello QAK

Davide Chirichella, Gabriele Doti, Daniele
Maccagnan

Ingegneria dei Sistemi Software, A.A. 2025/2026
Alma Mater Studiorum . Università di Bologna

July 09, 2026

1. Obiettivi dello Sprint 1

Obiettivo primario dello Sprint 1 è quello di transitare dall'architettura concettuale e dall'analisi dei requisiti maturate nello Sprint 0 verso la realizzazione di un primo **prototipo eseguibile del core-business** del sistema, da poter sottoporre a verifica formale e mostrare al committente.

In ottica di rigorosa **Separation of Concerns** e conformemente a un approccio incrementale guidato dal rischio (**risk-driven**), lo sviluppo si concentra sulla correttezza comportamentale, sulle logiche di sincronizzazione e sul protocollo di comunicazione dell'orchestratore principale (`cargoservice`). Occorre considerare che le funzionalità trattate presuppongono la cooperazione con dispositivi fisici di campo (sensori sonar per la rilevazione della pedana), attuatori (display `IOPort`, LED di segnalazione, marker laser di etichettatura) e unità robotiche mobili che non sono ancora state sviluppate né integrate sull'hardware finale (es. Raspberry Pi PicoW o robot differenziale reale).

Per isolare il collaudo della logica applicativa da incertezze tecnologiche e ritardi di fornitura hardware, nello Sprint 1 si ricorre all'introduzione di **componenti mock** progettati ad hoc. Tali entità simulano in modo temporizzato, deterministico e ripetibile il protocollo di interazione dei sottosistemi mancanti, permettendo di validare la Macchina a Stati Finiti (FSM) del servizio marittimo in tutte le condizioni operative e di eccezione.

2. Sottoinsieme di Requisiti Considerati

In continuità con quanto stabilito nel piano di lavoro dello Sprint 0, il presente Sprint prende in esame il sottoinsieme di requisiti ad alta priorità che governano l'ammissione dei container, la gestione delle risorse di stiva e il coordinamento del ciclo di movimentazione:

- **Requisiti `cargoservice` (Orchestratore di Dominio):**
 - Deve essere in grado di ricevere e processare le richieste di carico (`load_request`) inviate dai clienti tramite il pulsante dell'interfaccia di ingresso (`ioport`).
 - Deve rifiutare immediatamente la richiesta, inviando una risposta di rinvio temporaneo (`load_retrylater`), qualora l'area di ingresso (`IOPort`) sia attualmente ingombrata da un container oppure se il sistema si trovi in stato di allarme e fuori servizio (`Out of service`).
 - Deve rifiutare definitivamente la richiesta (`load_refused`) quando la stiva (`hold`) risulta completamente satura (ovvero i 4 slot di stoccaggio `slot1` - `slot4` sono tutti occupati).
 - In caso di esito positivo e risorse disponibili, deve commutare il proprio stato logico in **engaged**, riservare atomica una cella libera in stiva e restituire al cliente la risposta `load_accepted(slotID)` indicante il codice dello slot allocato.
 - Per tutta la durata in cui il sistema permane nello stato **engaged**, deve comandare all'attuatore LED di lampeggiare in modo continuo.

- ▶ Al rilascio di `load_accepted`, deve attendere che il cliente posizioni fisicamente il container sulla pedana entro un intervallo temporale limite prestabilito (es. 30 secondi); scaduto tale termine senza rilevamento, deve sbloccare la riserva e retrocedere allo stato **disengaged**.
- ▶ Una volta rilevato il container, deve coordinare l'unità robotica per movimentare il carico dall'IOPort verso la postazione di etichettatura (`slot5`), comandare al dispositivo marker l'applicazione del codice identificativo e, a marcatura ultimata, ordinare al robot il deposito nello slot riservato, concludendo il ciclo e ripristinando lo stato **disengaged**.
- **Requisiti dei Collaboratori e Interfacce di Bordo:**
 - ▶ `ioport`: Deve fungere da interfaccia verso l'utente, emettendo le richieste di carico e visualizzando sul display lo stato dell'impianto ("Service working" o "Out of service") nonché l'esito della prenotazione.
 - ▶ `sonar` e **allarmi**: Il sensore deve misurare costantemente la distanza dalla pedana. Qualora rilevi una distanza $D > D_{\text{FREE}}$ per un tempo continuativo di almeno 3 secondi, deve segnalare una condizione di possibile guasto, forzando l'impianto in stato **Out of service** e bloccando l'ammissione di nuove richieste.
 - ▶ `markerdevice` e `led`: Devono operare come attuatori passivi in grado di ricevere comandi di esecuzione e, nel caso del marker, notificare il completamento dell'etichettatura.

3. Architettura dello Sprint 0 (Riferimento e Punto di Partenza)

L'analisi svolta nello Sprint 0 ha fornito il modello di dominio di riferimento (`/sprint0/src/cargosystem.qak`), individuando le macro-entità del problema e definendo i primi contratti di interazione in linguaggio QAK.

Nello specifico, l'architettura di partenza era caratterizzata dai seguenti elementi:

1. **Decomposizione nei macro-sottosistemi:** Sono stati identificati gli attori principali del dominio: l'orchestratore `cargoservice`, l'interfaccia utente `ioport`, il movimentatore `cargorobot` e i dispositivi di campo `sonar`, `markerdevice` e `led`, oltre alla struttura logica `hold` per la mappatura degli slot.
2. **Topologia concettuale distribuita:** Il modello originario ipotizzava una ripartizione su 4 contesti di esecuzione fisicamente o logicamente distinti (`ctxcargoservice`, `ctxioport`, `ctxrobot`, `ctxdevices`), riflettendo l'intenzione di distribuire i componenti su nodi di calcolo eterogenei.
3. **Interazione di Ammissione:** Era già stato codificato il protocollo di base di tipo **Request/Reply** per l'invio della richiesta di carico (`load_request` → `load_accepted` / `load_retrylater` / `load_refused`).
4. **Astrazione del Comportamento (FSM preliminare):** Nel modello dello Sprint 0, il `cargoservice` era descritto tramite una Macchina a Stati Finiti altamente astratta, che sostava in un unico stato generico di lavoro (`work`) e utilizzava

semplici commenti testuali o stampe a video (`println`) per indicare le intenzioni di coordinamento del robot e della stiva.

Il compito analitico dello Sprint 1 consiste nel colmare l'abstraction gap rimasto aperto, trasformando le interazioni abbozzate e i commenti informali dello Sprint 0 in un protocollo di esecuzione rigoroso, formalmente collaudabile e resiliente ai guasti e ai ritardi ambientali.

4. Analisi Approfondita del Problema (Stile Accademico)

Per giungere alla definizione di un'architettura logica esecutiva e di un prototipo di alta qualità, è indispensabile analizzare nel dettaglio le problematiche ingegneristiche emerse durante la transizione dai requisiti ai contratti software. Seguendo l'approccio analitico adottato in progetti accademici di riferimento per sistemi concorrenti e reattivi, si valutano di seguito le diverse alternative progettuali e le relative giustificazioni.

4.1. Analisi del Core Business: Cargoservice e Stiva (Hold)

4.1.1. Gestione dello Stato della Stiva: Volatile vs Persistente

Un primo nodo decisionale critico concerne la rappresentazione e la memorizzazione dello stato di occupazione dei 4 slot della stiva marittima e dell'area di etichettatura (`slot5`). Durante l'analisi si sono contrapposte due macro-soluzioni architetturali:

1. **Persistenza Relazionale/NoSQL o su File System:** L'adozione di un database esterno (es. SQLite, PostgreSQL) o di un sistema di log persistente garantisce che lo stato della stiva sopravviva ad eventuali riavvii accidentali dell'applicazione, permettendo al contempo di storicizzare le associazioni tra codice identificativo del container e slot allocato.
2. **Struttura Dati Volatile in Memoria (Array/Mappe in RAM JVM):** La modellazione della stiva come attore locale o componente in memoria dell'orchestratore, provvisto di strutture di array (es. `intArrayOf(0, 0, 0, 0)`), assicura tempi di accesso istantanei, assenza di overhead di I/O e una semplicità algoritmica ottimale.

Giustificazione Ingegnistica

Per gli obiettivi specifici dello Sprint 1 — centrati sulla verifica formale della logica di coordinamento della FSM e del protocollo di smistamento — l'introduzione di un database esterno rappresenterebbe una complessità accidentale prematura (**over-engineering**). I requisiti attuali non impongono la persistenza a fronte di crash di sistema o la storicizzazione di lungo periodo; si richiede bensì una verifica transazionale rapida e atomica sulla disponibilità degli spazi. Pertanto, si opta per la realizzazione della stiva tramite un attore QAK dedicato (`hold`) dotato di memoria volatile. Tale astrazione isola l'algoritmo di ricerca dello slot

libero (`getFreeSlot()`) dal supporto fisico di memorizzazione, lasciando aperta la possibilità di iniettare un adapter verso un database persistente nelle fasi di rilascio finali.

4.1.2. Protocollo di Ammissione e Accodamento Richieste

Un ulteriore quesito analitico riguarda il comportamento da adottare quando il servizio riceve una richiesta di carico (`load_request`) mentre si trova già impegnato nella gestione di un ciclo operativo precedente o quando sussistono condizioni ambientali avverse. Si pongono due alternative di gestione del traffico:

1. **Bufferizzazione e Accodamento Applicativo:** Il servizio accoglie le richieste in arrivo, le inserisce in una coda FIFO applicativa e le processa sequenzialmente via via che la stiva o la pedana si liberano.
2. **Rifiuto Immediato o Rinvio Non-Bloccante:** Il servizio analizza istantaneamente lo stato dell'infrastruttura e, in caso di indisponibilità temporanea o permanente, emette all'istante una risposta di rinvio (`load_retrylater`) o rifiuto (`load_refused`), lasciando al cliente l'onere di riprovare in futuro.



Giustificazione Ingegneristica

Le risposte del committente maturate durante lo Sprint 0 hanno chiarito in modo inequivocabile che **le richieste non devono essere bufferizzate**. Nel paradigma QAK, ogni attore è dotato nativamente di una coda di messaggi in ricezione gestita dal runtime della JVM; tuttavia, per rispettare il vincolo contrattuale di non-accodamento logico, l'orchestratore `cargoservice` deve processare i messaggi in modo continuo senza rimanere bloccato in attese lunghe. La scelta ingegneristica ricade sul mantenimento rigoroso del pattern **Request/Reply**. La risposta `load_retrylater` viene emessa immediatamente valutando una guardia booleana atomica che compendia le tre precondizioni di inibizione: sistema già impegnato (`CargoState = "engaged"`), impianto in allarme (`!ServiceWorking`) o pedana di ingresso già occupata da un altro corpo (`IOPortOccupied`). Ciò evita lo spreco di risorse in bufferizzazione applicativa e garantisce reattività real-time alle interfacce utente.

4.1.3. Interazione con la Stiva (`hold`): Transazione a Due Fasi

Nel momento in cui una richiesta di carico supera le verifiche di ammissibilità preliminare, il servizio deve allargare il controllo alla disponibilità effettiva della stiva. In termini architetturali, si analizza se `cargoservice` debba conoscere direttamente la mappa delle celle oppure interrogare un'entità terza. L'incapsulamento della stiva in un attore separato (`hold`) impone la definizione di un protocollo di sincronizzazione sicuro per evitare **race conditions** o allocazioni multiple sul medesimo slot.

Giustificazione Ingegneristica

Si è optato per un protocollo transazionale a due fasi basato sullo scambio di messaggi sincroni `Request get_slot` / `Reply slot_reserved` (o `hold_full`). L'orchestratore non risponde subito all'utente con una conferma, ma inoltra la richiesta di allocazione all'attore `hold`. Solo alla ricezione della risposta asincrona di riserva confermata (`slotReserved(SLOTID)`), l'orchestratore commuta il proprio stato in **engaged**, attiva il lampeggio del LED e inoltra al cliente il messaggio formale `load_accepted(slotX)`. Questa separazione garantisce la coerenza transazionale dello stato anche in scenari concorrenti e disaccoppia la logica di smistamento dalla struttura dati interna della stiva.

4.1.4. Interazione e Coordinamento tra Orchestratore, Robot e Marker

La movimentazione del container dalla pedana al punto di marcatura (`slot5`) e infine alla cella di destinazione richiede una stretta cooperazione tra `cargoservice`, `cargorobot` e `markerdevice`. L'analisi del problema solleva una questione di progettazione fondamentale: **chi detiene il controllo del flusso di lavoro?**

1. **Robot Autonomo (Smart Robot / Choreography):** L'orchestratore si limita a comunicare al robot l'identificativo dello slot finale. Il robot ingloba la logica di dominio, recandosi in autonomia allo `slot5`, richiedendo la marcatura e poi procedendo allo stoccaggio finale.
2. **Orchestratore Centrale (Single Point of Control / Orchestration):** Il `cargoservice` mantiene il totale controllo della sequenza applicativa. Il robot viene degradato a mero esecutore cinetico di comandi elementari di traslazione (`move to X`), mentre l'orchestratore supervisiona le transizioni di stato e coordina esplicitamente il marker.

Giustificazione Ingegneristica

L'adozione del modello di **Orchestrazione Centrale (Single Point of Control)** è la soluzione vincente per mantenere una chiara **Separation of Concerns**. Delegare le regole di business navale (l'obbligo di etichettatura preventiva in `slot5`) all'unità robotica avrebbe violato la coesione architetturale, rendendo il robot riutilizzabile con difficoltà in altri contesti industriali. Nello Sprint 1, l'orchestratore `cargoservice` invia una richiesta `robot_move(slot5)` a `cargorobotmock`, attende il messaggio di completamento (`robot_done`), invia una richiesta `mark_container` a `markerdevice`, ne attende l'esito (`marking_done`) e invia infine la seconda movimentazione `robot_move(slotX)`. Questo srotolamento asincrono (**unrolling**) elimina ogni blocco sul thread principale e garantisce la tracciabilità millimetrica di ogni fase di lavoro nei log di audit.

4.1.5. Gestione Reattiva degli Allarmi dal Sonar e Routing Asincrono

Uno dei problemi teorici e pratici più complessi affrontati in questo Sprint riguarda la reattività dell'infrastruttura a fronte di eventi ambientali imprevisti. In conformità ai requisiti ufficiali, il sensore sonar è un puro dispositivo di campo che emette esclusivamente eventi broadcast (`sonardata : distance(D)`), senza conoscere l'orchestratore né emettere dispatch di allarme. È l'orchestratore `cargoservice` che analizza internamente le letture: se rileva $D > D_{\text{FREE}}$ per un tempo prolungato, imposta `ServiceWorking = false` (Out of service); se $D \leq D_{\text{FREE}}$, ripristina `ServiceWorking = true` (Service working); se $D < 50$, riconosce la presenza del container sull'IOPort. Il problema analitico risiede nel come consentire all'orchestratore di assorbire queste notifiche asincrone e di aggiornare le proprie variabili di stato (`IOPortOccupied` , `ServiceWorking`) **senza perdere la memoria dello stato logico corrente** (ossia se l'impianto si trovi in `disengaged` o `engaged`).



Giustificazione Ingegneristica

Per scongiurare la duplicazione di decine di righe di transizione condizionale all'interno di ogni singolo stato operativo, è stato ideato uno **stato di routing asincrono centralizzato** denominato `returnToState` . Qualunque evento ambientale ricevuto durante gli stati di riposo o attesa provoca una transizione immediata verso lo stato di decodifica (`handle_sonar`), al termine del quale il motore scatta verso `returnToState` . Questo stato è privo di codice applicativo e agisce come un selettore di flusso puramente logico: esamina la variabile di memoria `CargoState` e ri-direziona l'attore nello stato `engaged` oppure `disengaged` . Si ottiene così una resilienza architetturale completa agli **interrupt** senza sporcare la logica di business principale.

4.1.6. Resilienza e Timeout di Deposito (`handle_deposit_timeout`)

L'ultimo aspetto analizzato concerne il comportamento da adottare nel caso limite in cui un cliente, dopo aver inoltrato una richiesta e aver ricevuto l'accettazione formale `load_accepted(slotID)` , non provveda a depositare materialmente il container sulla pedana di ingresso. In assenza di un meccanismo di guardia temporale, il sistema rimarrebbe imprigionato a tempo indeterminato nello stato `engaged` : lo slot riservato in stiva resterebbe bloccato, il LED continuerebbe a lampeggiare e nessun altro cliente potrebbe accedere al servizio.



Giustificazione Ingegneristica

Si è resa obbligatoria l'integrazione di un meccanismo di recovery basato su un **timeout temporale non-bloccante**. Entrato nello stato di attesa del carico (`engaged`), l'attore imposta un timer nativo QAK (`whenTime 30000`). Se entro 30 secondi il sonar non notifica una distanza inferiore a 50 cm

(presenza del container all'IOPort), la transizione temporale innesca lo stato di compensazione `handle_deposit_timeout`. Qui, l'orchestratore emette un dispatch `free_slot($ReservedSlotId)` verso la stiva per sbloccare la cella, comanda lo spegnimento del LED, reimposta `CargoState = "disengaged"` e ritorna operativo per nuove richieste. Questo pattern di **compensazione transazionale** garantisce che l'impianto mantenga la propria disponibilità (**liveness**) anche a fronte di client difettosi o negligenti.

4.2. Analisi della Topologia e Disposizione nei Contesti QAK

4.2.1. Contesto Unico vs Contesti Distribuiti

In continuità con quanto sollevato nello Sprint 0, è necessario stabilire la disposizione fisica e logica dei 7 attori individuati (`cargoservice`, `hold`, `sonarmock`, `markerdevice`, `ledmock`, `ioportmock`, `cargorobotmock`). Si mettono a confronto due opzioni di deployment per il prototipo di verifica:

1. **Mantenimento dei 4 Contesti Distribuiti (Sprint 0):** I componenti vengono istanziati su processi o JVM differenti all'interno della stessa macchina o su reti di test (porte TCP `8050`, `8051`, `8052`, `8053`), comunicando via socket di rete.
2. **Aggregazione in un Contesto Unico Locale (`ctxprototype` :8050):** Tutti i 7 attori vengono compattati ed eseguiti all'interno del medesimo ambiente QAK su una sola JVM locale sulla porta `8050`.

PARAMETRO	SPRINT 0 (ANALISI)	SPRINT 1 (PROTOTIPO)	MOTIVAZIONE INGEGNERISTICA
Topologia	4 Contesti elementari distanti tra loro	1 Contesto Unico (<code>ctxprototype</code> :8050)	Focalizzazione sulla correttezza logica della FSM senza introdurre complessità di rete premature.
Meccanismo di Comunicazione	TCP/IP tra nodi di rete separati	Scambio messaggi locale (in memoria, stessa JVM)	Elimina latenza e overhead di serializzazione, rendendo il debug formale e immediato.
Preparazione per Sprint 2	N/A	Architettura multi-contesto già archiviata in <code>uTils/prototype/</code>	La separazione in contesti fisici distribuirà gli attori sui nodi reali nel prossimo Sprint.



Giustificazione Ingegneristica

La decisione di adottare il **Contesto Unico** (`ctxprototype` :8050) nello Sprint 1 risponde a un imperativo metodologico: **separare la complessità computazionale dalla complessità di distribuzione**. Se si fossero eseguiti i collaudi iniziali su 4 contesti di rete separati, eventuali fallimenti di comunicazione o stalli della FSM avrebbero potuto essere imputabili a fattori infrastrutturali esterni (latenze TCP, fallimenti di serializzazione delle stringhe Prolog, blocchi del sistema operativo o configurazioni di rete errate) anziché a difetti intrinseci alla logica di business.

L'aggregazione in un contesto unico locale consente un debug formale, immediato e deterministico. L'architettura distribuita multi-contesto delineata nello Sprint 0 non è stata affatto abbandonata: è stata accuratamente implementata e archiviata come libreria di riferimento nella cartella `utils/prototype/`, pronta per essere schierata senza modifiche alla business logic non appena l'hardware reale verrà collegato nello Sprint 2.

5. Riepilogo dei Messaggi del Sistema

Per garantire chiarezza formale e fungere da contratto per la successiva implementazione, la tabella seguente riassume l'intero set di messaggi (Request, Reply, Dispatch, Event) definiti per governare il prototipo dello Sprint 1 all'interno del contesto `ctxprototype` :

NOME MESSAGGIO	TIPO QAK	MITTENTE	DESTINATARIO	PAYLOAD / SEMANTICA
<code>load_request</code>	Request	<code>ioport</code> / <code>ioportmock</code>	<code>cargoservice</code>	<code>loadRequest(none)</code> Richiesta di ammissione del container.
<code>load_accepted</code>	Reply	<code>cargoservice</code>	<code>ioport</code> / <code>ioportmock</code>	<code>loadAccepted(SLOTID)</code> Richiesta accolta, slot riservato.
<code>load_retrylater</code>	Reply	<code>cargoservice</code>	<code>ioport</code> / <code>ioportmock</code>	<code>loadRetryLater(none)</code> Rinvio temporaneo (occupato o out of service).
<code>load_refused</code>	Reply	<code>cargoservice</code>	<code>ioport</code> / <code>ioportmock</code>	<code>loadRefused(none)</code> Rifiuto definitivo (stiva piena).
<code>get_slot</code>	Request	<code>cargoservice</code>	<code>hold</code>	<code>getSlot(none)</code> Interrogazione alla

				stiva per allocazione.
<code>slot_reserved</code>	Reply	<code>hold</code>	<code>cargoservice</code>	<code>slotReserved(SLOTID)</code> Identificativo 1-indexed della cella libera.
<code>hold_full</code>	Reply	<code>hold</code>	<code>cargoservice</code>	<code>holdFull(none)</code> Notifica di esaurimento dei 4 slot.
<code>free_slot</code>	Dispatch	<code>cargoservice</code>	<code>hold</code>	<code>freeSlot(SLOTID)</code> Comando di rilascio cella per timeout o scarico.
<code>sonardata</code>	Event	<code>sonar</code> / <code>sonarmock</code>	Broadcast / <code>cargoservice</code>	<code>distance(D)</code> Misura in cm della distanza dalla pedana o ostacolo.
<code>led_ctrl</code>	Dispatch	<code>cargoservice</code>	<code>led</code> / <code>ledmock</code>	<code>ledCmd(CMD)</code> Comando attuatore (<code>on</code> , <code>off</code> , <code>blink</code>).
<code>robot_move</code>	Request	<code>cargoservice</code>	<code>cargorobotmock</code>	<code>robotMove(TARGET)</code> Comando di traslazione verso <code>slot5</code> o <code>slotX</code> .
<code>robot_done</code>	Reply	<code>cargorobotmock</code>	<code>cargoservice</code>	<code>robotDone(none)</code> Notifica di arrivo a destinazione.
<code>mark_container</code>	Request	<code>cargoservice</code>	<code>markerdevice</code>	<code>markContainer(none)</code> Avvio operazione laser di etichettatura.
<code>marking_done</code>	Reply	<code>markerdevice</code>	<code>cargoservice</code>	<code>markingDone(none)</code> Conferma di avvenuta etichettatura.

6. Nuova Architettura Logica (Sprint 1)

L'integrazione delle decisioni analitiche finora discusse porta alla definizione di una **Nuova Architettura Logica** per il prototipo dello Sprint 1, illustrata formalmente e sintetizzata nel modello di deployment unico locale.

All'interno dell'unico ambiente esecutivo `ctxprototype` (in esecuzione sull'host `localhost` in ascolto sulla porta TCP `8050`), operano in regime concorrente e asincrono i 7 attori del sistema:

- `cargoservice`: L'attore orchestratore, cuore applicativo che implementa la FSM di controllo, valuta le guardie booleane di ammissione e governa l'avanzamento sequenziale tramite le risposte dei collaboratori.
- `hold`: L'attore di stoccaggio, gestore transazionale della memoria volatile degli slot navali.
- `sonarmock` e `ioportmock`: I generatori di stimoli e client intelligenti. Il `sonarmock` diffonde eventi ambientali di presenza (`sonardata(30)`) ed emula guasti temporanei del sensore (`setServiceStatus(outofservice)`), mentre `ioportmock` esegue uno script temporizzato di richieste di carico per saggiare la reattività del servizio sia in condizioni normali che di allarme.
- `markerdevice`, `ledmock` e `cargorobotmock`: Gli attuatori simulati. Ricevono comandi operativi dal servizio e simulano il tempo fisico delle operazioni industriali tramite ritardi controllati (`delay`), restituendo la notifica di completamento per permettere alla FSM di avanzare al passo successivo.

Questa architettura logica rappresenta un superamento qualitativo ed esecutivo del modello dello Sprint 0: elimina ogni vaghezza interpretativa, sostituisce i commenti informali con interazioni rigorose tra attori e costituisce una base formale verificabile end-to-end tramite test automatizzati JUnit prima della discesa sul campo fisico prevista per lo Sprint 2.

7. Progettazione e Scelte Implementative (`cargoservice`)

La progettazione dell'orchestratore si basa sul principio di **Single Point of Control**: `cargoservice` coordina tutte le fasi di ammissione, interroga la stiva e gestisce le movimentazioni senza delegare la logica di controllo ad altri entità. Di seguito vengono analizzate le scelte progettuali attuate nel passaggio da Sprint 0 a Sprint 1.

7.1. Sdoppiamento dello Stato di Attesa (`work` → `disengaged` / `engaged`)

Nello Sprint 0 il sistema sosta in un generico stato `work`. Per aderire fedelmente alla terminologia contrattuale (“**considera il sistema come engaged quando gestisce un carico... altrimenti disengaged**”), lo stato `work` è stato eliminato in favore di due stati espliciti:

- `disengaged`: Sistema in inattività logica, LED spento e pronto a ricevere nuove richieste.

- `engaged` : Sistema impegnato, LED in lampeggio e precondizioni di ammissione attive.

7.2. Guardia Booleana sulle Precondizioni di Carico

In fase di ammissione (`handle_load_request`), le regole decisionali abbozzate a parole nello Sprint 0 sono state tradotte in un'asserzione di guardia valutata deterministicamente:

```
if [# CargoState = "engaged" || !ServiceWorking || IOPort0occupied #] {
    replyTo load_request with load_retrylater : loadRetryLater(none)
} else {
    request hold -m get_slot : getSlot(none)
}
```

Se l'impianto è occupato, in allarme o la pedana non è sgombra, la richiesta viene deviata senza bloccare l'attore.

7.3. Srotolamento Asincrono (“Unrolling”) del Ciclo Operativo

Nello Sprint 0 l'intero processo di movimentazione, etichettatura e stoccaggio era compresso in commenti inline. Nello Sprint 1, per rispettare la natura non bloccante degli attori, questo flusso è stato srotolato in **7 stati sequenziali e reattivi**:

1. `wait_for_slot` : Attesa asincrona della conferma di disponibilità da parte della stiva (`hold`).
2. `accept_request` / `refuse_request` : Smistamento finale della risposta verso la `ioport`.
3. `handle_sonar` : Ricezione degli eventi `sonardata` per certificare il posizionamento fisico del container.
4. `do_robot_job` : Richiesta di prelievo e trasporto al robot verso la zona di etichettatura (`slot5`).
5. `mark_container` : Interrogazione al `markerdevice` per l'applicazione del codice laser.
6. `move_to_reserved_slot` : Ordine finale al robot per il deposito nella cella riservata.
7. `finish_job` : Risoluzione della transazione, spegnimento del LED e ripristino di `disengaged`.

7.4. Gestione degli Interrupt e Routing Asincrono (`returnToState`)

Un punto aperto di fondamentale importanza riguarda la gestione di eventi asincroni improvvisi, come gli eventi del sonar (`sonardata : distance(D)` che comportano variazioni di `IOPort0occupied` o commutazione di `ServiceWorking`). L'attore deve poter aggiornare le variabili ambientali senza perdere la memoria logica del lavoro in corso. A tale scopo è stato progettato lo stato di routing `returnToState` :

```
State returnToState {}
Goto engaged if [# CargoState = "engaged" #] else disengaged
```

Questo meccanismo agisce da trampolino: dopo aver processato l'evento, il sistema valuta la variabile di stato `CargoState` e rientra in `engaged` (se era nel mezzo di un ciclo) o in `disengaged` (se era in attesa), eliminando la ridondanza e impedendo stalli logici.

7.5. Timeout di Sicurezza sul Deposito (`handle_deposit_timeout`)

Per evitare che il sistema rimanga congelato indefinitamente se un cliente, dopo aver ricevuto `load_accepted`, non deposita il container, è stata introdotta una transizione di tempo di 30 secondi nello stato `engaged`. Allo scadere del timeout, l'orchestratore attiva `handle_deposit_timeout`, invia alla stiva un messaggio di rilascio (`free_slot`), spegne il LED e torna allo stato `disengaged`.

8. Confronto Tabellare degli Stati (`cargoservice`)

STATO QAK	SPRINT 0	SPRINT 1	SIGNIFICATO E MODIFICA ARCHITETTURALE
<code>s0</code>	Presente	Presente	Inizializzazione variabili di stato e connessioni.
<code>work</code>	Presente (Attesa)	Eliminato	Sostituito per chiarezza terminologica da due stati distinti.
<code>disengaged</code>	Assente	Nuovo	Sistema libero. Il LED è spento e si attendono richieste.
<code>engaged</code>	Assente	Nuovo	Sistema occupato. Il LED lampeggia, preconditioni attive.
<code>handle_load_request</code>	Solo commenti	Logica Es-eguibile	Valida la guardia booleana e interroga la stiva (<code>get_slot</code>).
<code>wait_for_slot</code>	Assente	Nuovo	Attesa asincrona e non-bloccante della risposta dall'hold.
<code>accept_request</code> / <code>refuse_request</code>	Assenti (inline)	Nuovi	Smistamento formale delle risposte al cliente (<code>ioport</code>).
<code>handle_sonar</code> / <code>update_service</code>	Assenti	Nuovi	Gestione reattiva degli eventi di presenza e guasto sonar.
<code>returnToState</code>	Assente	Nuovo	Stato di routing per tornare a <code>engaged</code> / <code>disengaged</code> dopo un interrupt.
<code>do_robot_job</code>	Solo commento	Nuovo	Richiesta al robot di prelievo e trasporto verso <code>slot5</code> .

<code>mark_container</code>	Solo commento	Nuovo	Attesa completamento etichettatura da parte del marker.
<code>move_to_reserved_slot</code>	Solo commento	Nuovo	Richiesta di deposito finale nello slot riservato dall'hold.
<code>finish_job</code>	Assente	Nuovo	Chiusura transazione, spegnimento LED, ritorno a <code>disengaged</code> .
<code>handle_deposit_timeout</code>	Assente	Nuovo	Scadenza di 30s. Libera lo slot in stiva (<code>free_slot</code>).

9. Analisi Puntuale del Codice del Prototipo Sprint 1

Per mostrare in modo esaustivo e trasparente come le scelte architetturali discusse siano state concretamente realizzate, di seguito viene riportato l'intero codice QAK del prototipo dello Sprint 1 (`/sprint1/prototype/codice_con_tutti_i_componenti.qak`), analizzato per sezioni logiche (snippet di codice accompagnati da analisi critica).

9.1. Dichiarazione del Sistema, Contesto e Messaggistica

```
System cargosystem

//INTERAZIONI TRA COMPONENTI

// Customer ↔ CargoService
Request load_request    : loadRequest(none)
Reply  load_accepted    : loadAccepted(SLOTID) for load_request
Reply  load_retrylater   : loadRetryLater(none) for load_request
Reply  load_refused      : loadRefused(none) for load_request

// CargoService ↔ Hold
Request get_slot        : getSlot(none)
Reply  slot_reserved    : slotReserved(SLOTID) for get_slot
Reply  hold_full        : holdFull(none) for get_slot
Dispatch free_slot      : freeSlot(SLOTID)

// Aggiornamenti Sensori e Stato
Event  sonardata         : distance(D) // Emesso dal sonar
Dispatch led_ctrl        : ledCmd(CMD)

// CargoService ↔ Robot
Request robot_move : robotMove(TARGET)
Reply  robot_done  : robotDone(none) for robot_move

// CargoService ↔ MarkerDevice
Request mark_container : markContainer(none)
Reply  marking_done    : markingDone(none) for mark_container

//CONTESTI
```

```
Context ctxprototype ip [host="localhost" port=8050] // Contesto unico del
prototipo per lo Sprint 1
```

Analisi del Blocco 1: La prima sezione del modello definisce i confini di interazione e l'infrastruttura di comunicazione.

- **Distinzione semantica dei messaggi:** In accordo con le buone pratiche di ingegneria dei protocolli, si distingue rigorosamente tra comunicazioni binarie sincrone/correlate (`Request` / `Reply`), comandi asincroni diretti (`Dispatch` , utilizzati ad esempio per liberare lo slot con `free_slot` o per accendere/spegnere il LED con `led_ctrl`) e notifiche broadcast ambientali (`Event` , impiegato dal sonar per diffondere la misura della distanza `sonardata`).
- **Contesto Unico:** La dichiarazione di `ctxprototype` sulla porta `8050` vincola tutti i 7 attori del sistema a eseguire nella stessa JVM. Come argomentato nell'analisi, ciò elimina il rumore infrastrutturale e le latenze TCP durante i test di correttezza della business logic.

9.2. Orchestratore (`cargoservice`): Inizializzazione e Stati di Riposo

```
QActor cargoservice context ctxprototype {
  [#
    var IOPortOccupied = false
    var ServiceWorking = true
    var CargoState      = "disengaged"
    var ReservedSlotId = -1
  #]

  State s0 initial {
    println("cargoservice | STARTED") color magenta
  }
  Goto disengaged

  State disengaged {
    println("cargoservice | DISENGAGED: waiting for requests...") color
blue
  }
  Transition t0
    whenRequest load_request → handle_load_request
    whenEvent   sonardata    → handle_sonar

  State engaged {
    println("cargoservice | ENGAGED: waiting for container deposit...")
color blue
  }
  Transition t0
    whenTime    30000          → handle_deposit_timeout
    whenRequest load_request   → handle_load_request
    whenEvent   sonardata      → handle_sonar
}
```

Analisi del Blocco 2:

- **Variabili del Modello di Memoria:** Nel blocco Kotlin `[# ... #]`, l'attore inizializza le variabili di stato interne che governano la Macchina a Stati Finiti. Il flag `IOPortOccupied` tiene traccia dell'ingombro fisico all'IOPort, `ServiceWorking` memorizza lo stato di salute dell'impianto (interrotto/funzionante), mentre `CargoState` ed esplicitamente il lessico del committente ("engaged/disengaged").
- **Sdoppiamento dell'Attesa:** A differenza dello Sprint 0 dove esisteva un solo stato `work`, qui si osserva la chiara separazione tra `disengaged` (nessuna operazione in corso) ed `engaged` (carico autorizzato e in attesa del container). In entrambi gli stati, l'attore mantiene la reattività verso le richieste di carico, gli eventi del sonar e gli allarmi di guasto.

9.3. Orchestratore (`cargoservice`): Ammissione e Smistamento Richieste

```
//GESTIONE RICHIESTE

State handle_load_request {
    printCurrentMessage color yellow

    // Verifica precondizioni per accettare la richiesta
    if [# CargoState = "engaged" || !ServiceWorking || IOPortOccupied
#] {
        replyTo load_request with load_retrylater : loadRetryLater(none)
    } else {
        // Interroga la Hold per uno slot libero
        request hold -m get_slot : getSlot(none)
    }
}

Transition t0
    whenReply slot_reserved → accept_request
    whenReply hold_full     → refuse_request

State accept_request {
    onMsg(slot_reserved : slotReserved(ID)) {
        [#
            ReservedSlotId = payloadArg(0).toInt()
            CargoState      = "engaged"
        #]
        forward ledmock -m led_ctrl : ledCmd(blink)
        [# val SlotName = "slot$ReservedSlotId" #]
        replyTo load_request with load_accepted : loadAccepted($SlotName)
    }
}

Goto engaged

State refuse_request {
    replyTo load_request with load_refused : loadRefused(none)
}

Goto disengaged
```

Analisi del Blocco 3:

- **Guardia Booleana di Ammissione:** Lo stato `handle_load_request` implementa la logica di filtro critico dell'intero sistema. La condizione `if [# CargoState == "engaged" || !ServiceWorking || IOPortOccupied #]` valuta simultaneamente la logica di business, la sicurezza hardware e lo stato fisico della pedana. In caso negativo, il servizio emette immediatamente la risposta `load_retrylater` senza bloccare la propria esecuzione.
- **Transazione a due fasi con la Stiva:** Se le precondizioni sono soddisfatte, l'orchestratore non accetta ciecamente la richiesta, ma inoltra una richiesta `get_slot` all'attore `hold`. Solo alla ricezione della risposta asincrona `slot_reserved` (transizione verso `accept_request`), l'orchestratore alloca il `ReservedSlotId`, commuta lo stato in `engaged`, comanda all'attuatore LED di lampeggiare (`ledCmd(blink)`) e invia la conferma `load_accepted(slotX)` al cliente.

9.4. Orchestratore (`cargoservice`): Gestione Sonar, Allarmi e Routing Asincrono

```
//GESTIONE SONAR

State handle_sonar {
  onMsg(sonardata : distance(D)) {
    [# val Dist = payloadArg(0).toInt() #]

    // D < 50 indica presenza del container (IOPort occupata)
    if [# Dist < 50 #] {
      [# IOPortOccupied = true #]
    } else {
      [# IOPortOccupied = false #]
    }

    // D > DFREE (es. > 150) per un intervallo prolungato indica
condizione OUT OF SERVICE
    if [# Dist > 150 #] {
      [# ServiceWorking = false #]
      println("cargoservice | Sonar D > DFREE ($Dist) → System OUT
OF SERVICE!") color red
    } else {
      if [# !ServiceWorking #] {
        println("cargoservice | Sonar D ≤ DFREE ($Dist) →
System WORKING again!") color green
      }
      [# ServiceWorking = true #]
    }
  }
}

// Se il container viene posato durante lo stato engaged e il sistema è
operativo, il robot può iniziare
Goto do_robot_job if [# IOPortOccupied && CargoState == "engaged" &&
ServiceWorking #] else returnToState

State returnToState {}
Goto engaged if [# CargoState == "engaged" #] else disengaged
```

Analisi del Blocco 4:

- **Aggiornamento Dinamico della Scena e degli Allarmi:** Lo stato `handle_sonar` decodifica il payload dell'evento `distance(D)`. La soglia numerica (`Dist < 50`) mappa il concetto fisico di “presenza del container sulla pedana” nella variabile booleana `IOPortOccupied`. Parallelamente, la valutazione `Dist > 150` ($D > D_{FREE}$) commuta `ServiceWorking = false` (Out of service) o `true` (Service working), attenendosi rigorosamente al Protobook.
- **Innesco Reattivo o Routing:** Se la pedana diventa occupata, il sistema si trova nello stato logico di ingaggio e non vi sono allarmi attivi (`IOPortOccupied && CargoState = "engaged" && ServiceWorking`), la FSM scatta automaticamente verso l'avvio della movimentazione robotica (`do_robot_job`).
- **Stato di Routing (`returnToState`):** Costituisce un punto di eccellenza architetturale dello Sprint 1. Le letture sonar terminano con una transizione a `returnToState`. Questo stato privo di corpo agisce da switch logico: interroga `CargoState` e riporta l'attore in `engaged` o in `disengaged`. In questo modo si garantisce resilienza totale alle interruzioni esterne senza duplicare il codice di transizione nei singoli stati operativi.

9.5. Orchestratore (`cargoservice`): Coordinamento Robot, Etichettatura e Timeout

```
// GESTIONE ROBOT E MARKER

State do_robot_job {
    println("cargoservice | Container deposited! Moving to slot5...")
    color magenta
    request cargorobotmock -m robot_move : robotMove(slot5)
}
Transition t0
    whenReply robot_done → mark_container

State mark_container {
    println("cargoservice | At slot5. Asking markerdevice to mark...")
    color magenta
    request markerdevice -m mark_container : markContainer(none)
}
Transition t0
    whenReply marking_done → move_to_reserved_slot

State move_to_reserved_slot {
    println("cargoservice | Marked! Moving to slot$ReservedSlotId...")
    color magenta
    [# val TargetSlot = "slot$ReservedSlotId" #]
    request cargorobotmock -m robot_move : robotMove($TargetSlot)
}
Transition t0
    whenReply robot_done → finish_job

State finish_job {
```

```

        println("cargoservice | Job completed!") color green
        [# CargoState = "disengaged" #]
        forward ledmock -m led_ctrl : ledCmd(off)
    }
    Goto disengaged

    State handle_deposit_timeout {
        println("cargoservice | Deposit timeout! Freeing slot.") color red

        [# CargoState = "disengaged" #]
        forward hold -m free_slot : freeSlot($ReservedSlotId)
        forward ledmock -m led_ctrl : ledCmd(off)
    }
    Goto disengaged
}

```

Analisi del Blocco 5:

- **Srotolamento Asincrono (Unrolling):** Questo gruppo di stati incarna il superamento del commento monolitico dello Sprint 0. L'orchestratore governa la sequenza operativa inviando richieste asincrone ai collaboratori (`cargorobotmock` e `markerdevice`). Ogni transizione di avanzamento è strettamente subordinata alla ricezione della risposta di completamento (`whenReply robot_done` , `whenReply marking_done`), garantendo che il robot e il laser operino in sincronia rigorosa senza bloccare il thread dell'orchestratore.
- **Chiusura del Ciclo (`finish_job`):** Una volta deposto il container nello slot finale, l'attore reimposta `CargoState = "disengaged"` e invia il dispatch di spegnimento al LED (`ledCmd(off)`), riportando il sistema allo stato di riposo iniziale.
- **Gestione del Caso Limite (Timeout):** Lo stato di recovery `handle_deposit_timeout` entra in azione allo scadere dei 30 secondi nello stato `engaged` in assenza del container. Per garantire che le risorse non rimangano bloccate a causa dell'inadempienza del cliente, l'orchestratore invia un messaggio di compensazione alla stiva (`freeSlot($ReservedSlotId)`), spegne il LED e risistema l'infrastruttura per nuove richieste.

9.6. Collaboratore Simulato di Stiva (`hold`)

```

//HOLD MOCK

QActor hold context ctxprototype {
    [#
        var Slots = intArrayOf(0, 0, 0, 0)

        fun getFreeSlot(): Int {
            for (i in 0..3) {
                if (Slots[i] == 0) return i + 1
            }
            return -1
        }
    #]
}

```

```

State s0 initial {
    println("hold | STARTED") color green
}
Goto work

State work {}
Transition t0
    whenRequest get_slot → handle_get_slot
    whenMsg      free_slot → handle_free_slot

State handle_get_slot {
    [# val SlotId = getFreeSlot() #]

    if [# SlotId ≠ -1 #] {
        [# Slots[SlotId - 1] = 1 #]
        replyTo get_slot with slot_reserved : slotReserved($SlotId)
    } else {
        replyTo get_slot with hold_full : holdFull(none)
    }
}
Goto work

State handle_free_slot {
    onMsg(free_slot : freeSlot(ID)) {
        [# val id = payloadArg(0).toInt() #]
        [# Slots[id - 1] = 0 #]
        println("hold | Freed slot$id") color green
    }
}
Goto work
}

```

Analisi del Blocco 6:

- **Memoria Volatile Mappata:** In accordo con le considerazioni sui requisiti ad alta priorità, l'attore `hold` modella la stiva navale tramite una struttura dati in memoria ad alta efficienza (`Slots = intArrayOf(0,0,0,0)`).
- **Atomicità di Allocazione e Rilascio:** Alla ricezione della richiesta `get_slot`, l'attore esegue l'algoritmo lineare di ricerca `getFreeSlot()`. Se individua una cella vuota, ne marca lo stato a `1` e risponde con l'identificativo 1-indexed (`slotReserved`); se tutti i 4 slot sono pieni, risponde immediatamente con `holdFull`. Parallelamente, la gestione del dispatch `free_slot` permette di resettare la cella a `0` (indispensabile sia per il timeout di deposito che per futuri cicli di scarico).

9.7. Sensori e Attuatori Simulati (`sonarmock`, `markerdevice`, `ledmock`)

```

//SONAR MOCK

QActor sonarmock context ctxprototype {
    State s0 initial {

```

```

        delay 4000
        println("sonarmock | Container placed → Emitting sonardata(30)")
color cyan
        emit sonardata : distance(30)

        // Simulazione di una distanza  $D > D_{FREE}$  (es. 200 per  $> 3s$ ) →
Innesca Out of service
        delay 8000
        println("sonarmock | Sonar measures  $D > D_{FREE}$  (200). Emitting
sonardata(200)") color cyan
        emit sonardata : distance(200)

        // Simulazione del ripristino  $D \leq D_{FREE}$  (es. 100) → System working
        delay 5000
        println("sonarmock | Sonar measures  $D \leq D_{FREE}$  (100). Emitting
sonardata(100)") color green
        emit sonardata : distance(100)
    }
}

//MARKER MOCK

QActor markerdevice context ctxprototype {
    State s0 initial {
        println("markerdevice | STARTED") color green
    }
    Goto work

    State work {}
    Transition t0
        whenRequest mark_container → handle_mark

    State handle_mark {
        println("markerdevice | Marking container...") color cyan
        delay 1500
        println("markerdevice | Container marked!") color cyan
        replyTo mark_container with marking_done : markingDone(none)
    }
    Goto work
}

//LED MOCK

QActor ledmock context ctxprototype {
    State s0 initial { }
    Goto work

    State work {}
    Transition t0
        whenMsg led_ctrl → handle_cmd

    State handle_cmd {
        onMsg(led_ctrl : ledCmd(CMD)) {
            println("ledmock | LED is now ${payloadArg(0)}") color green
        }
    }
}

```

```

    }
  }
  Goto work
}

```

Analisi del Blocco 7:

- **Simulatore Sonar Dinamico:** Il `sonarmock` funge da generatore di stimoli per collaudare la reattività dell'orchestratore. Mediante una sequenza temporizzata (`delay`), emette in primo luogo l'evento di presenza container (`distance(30)`), e successivamente simula un guasto ambientale prolungato inviando `setServiceStatus(outofservice)`, seguito dal ripristino (`working`). Ciò dimostra che la FSM è in grado di gestire interrupt dinamici senza fallimenti.
- **Emulazione Attuatori:** Il `markerdevice` simula il tempo fisico di marcatura laser con un ritardo bloccante locale (`delay 1500`) restituendo la risposta di completamento al termine. Il `ledmock` funge da ricevitore puro per certificare visivamente nei log di debug che l'accensione, il lampeggio e lo spegnimento avvengano esattamente in corrispondenza delle transizioni logiche del servizio.

9.8. Client e Robot Simulati (`ioportmock`, `cargorobotmock`)

```

//IOPORT MOCK

QActor ioportmock context ctxprototype {
  State s0 initial {
    delay 1000
    println("ioportmock | Sending 1st load_request (should be accepted)")
    color cyan
    request cargoservice -m load_request : loadRequest(none)
  }
  Transition t0
    whenReply load_accepted → handle_accept
    whenReply load_retrylater → handle_retry
    whenReply load_refused → handle_refuse

  State handle_accept {
    printCurrentMessage color green

    // Aspettiamo che il sonarmock vada in outofservice (circa al ms
12000)
    delay 12000
    println("ioportmock | Sending 2nd load_request (should retrylater due
to outofservice)") color cyan
    request cargoservice -m load_request : loadRequest(none)
  }
  Transition t0
    whenReply load_retrylater → wait_and_retry
    whenReply load_accepted → handle_final_accept
    whenReply load_refused → handle_refuse

  State wait_and_retry {

```

```

        printCurrentMessage color yellow

        // Aspettiamo il ripristino del sonarmock (circa al ms 17000)
        delay 6000
        println("ioportmock | Sending 3rd load_request (should be accepted,
system recovered)") color cyan
        request cargoservice -m load_request : loadRequest(none)
    }
    Transition t0
        whenReply load_accepted → handle_final_accept
        whenReply load_retrylater → handle_retry
        whenReply load_refused → handle_refuse

    State handle_final_accept { printCurrentMessage color green }
    State handle_retry { printCurrentMessage color yellow }
    State handle_refuse { printCurrentMessage color red }
}

//CARGOROBOT MOCK

QActor cargorobotmock context ctxprototype {
    State s0 initial { }
    Goto work

    State work {}
    Transition t0
        whenRequest robot_move → handle_move

    State handle_move {
        delay 1500
        replyTo robot_move with robot_done : robotDone(none)
    }
    Goto work
}

```

Analisi del Blocco 8:

- **Scripting di Collaudo Integrato (`ioportmock`)**: Questo attore rappresenta un client intelligente che esegue uno script di collaudo end-to-end all'avvio del sistema. Invia 3 richieste in momenti strategici del ciclo di vita del prototipo: la prima trova il sistema libero ed operante (ricevendo `load_accepted`); la seconda viene inviata al millisecondo 12000, esattamente durante la simulazione di guasto del sonar (certificando che l'orchestratore risponda correttamente con `load_retrylater`); la terza avviene dopo il ripristino, confermando che il sistema è pienamente resiliente ed in grado di accogliere nuove richieste una volta risolta l'anomalia.
- **Astrazione delle Movimentazioni (`cargorobotmock`)**: Il robot simulato risponde al contratto di movimentazione (`robotMove`) con un tempo d'attesa parametrico (`delay 1500`), separando in modo netto le responsabilità tra chi decide dove deve andare il container (l'orchestratore) e chi esegue materialmente il moto (il robot, la cui navigazione su mappa verrà approfondita con il `basicrobot` nello Sprint successivo).

10. Piano di Test e Collaudo Automatizzato

L'impiego di QAK e la scomposizione modulare realizzata nello Sprint 1 forniscono la base per la verifica rigorosa dei requisiti. La suite di collaudo, tradotta in codice Kotlin con framework JUnit (`TestCargoServiceCore.kt`), agisce come un client esterno che si connette al contesto `ctxprototype` (porta 8050) e invia richieste strutturate secondo la semantica Prolog (`msg(load_request, ...)`).

L'esecuzione nel contesto unico garantisce un collaudo ripetibile e deterministico:

1. **Verifica Accettazione (T1):** A stiva disponibile e sistema `disengaged` , il test verifica l'effettiva ricezione della risposta `load_accepted` e la corretta prenotazione dello slot.
2. **Verifica Rinvio e Rifiuto (T2/T3):** Simula condizioni di stiva satura o pedana occupata, asserendo il ritorno di `load_retrylater` o `load_refused` .
3. **Resilienza agli Allarmi:** Verifica che la ricezione di eventi `sonardata` con $D > D_{\text{FREE}}$ (es. `distance(200)`) inneschi l'aggiornamento di `ServiceWorking = false` e il conseguente rinvio delle richieste (`load_retrylater`), e che il successivo ripristino con $D \leq D_{\text{FREE}}$ riporti la normale operatività tramite la transizione `returnToState` .

11. Conclusioni e Sviluppi Futuri

Traguardi Raggiunti e Roadmap per lo Sprint 2

In sintesi, le scelte progettuali dello Sprint 1 sono state guidate da criteri di modularità, chiarezza comportamentale e aderenza al lessico di dominio. L'architettura formalizzata garantisce la totale separazione tra la logica di controllo e le dipendenze fisiche esterne.

Il prototipo così validato costituisce la base solida ed estensibile per gli sviluppi dello Sprint 2, nei quali i collaboratori mock verranno progressivamente sostituiti dalle interfacce reali verso la web-gui, i sensori su Raspberry Pi e l'attuatore `cargorobot` .